What's Next

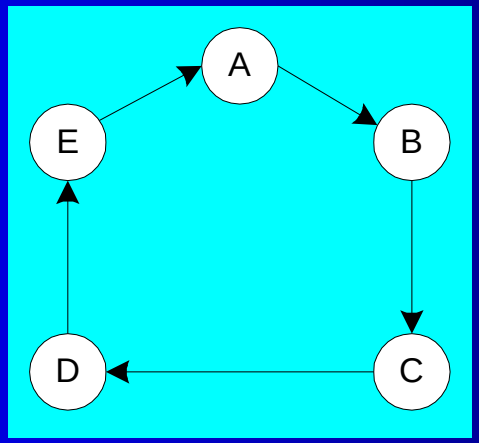
- Stack Frames
- **Recursion**
- INVOKE, ADDR, PROC, and PROTO
- Creating Multimodule Programs
- Advanced Use of Parameters (optional)
- Java Bytecodes (optional)

Recursion

- What is Recursion?
- Recursively Calculating a Sum
- Calculating a Factorial

What is Recursion?

- The process created when . . .
  - A procedure calls itself
  - Procedure A calls procedure B, which in turn calls procedure A
- Using a graph in which each node is a procedure and each edge is a procedure call, recursion forms a cycle:



Recursively Calculating a Sum

The CalcSum procedure recursively calculates the sum of an array of integers. Receives: ECX = count. Returns: EAX = sum

```
CalcSum PROC
    cmp ecx,0                ; check counter value
    jz L2                    ; quit if zero
    add eax,ecx              ; otherwise, add to sum
    dec ecx                  ; decrement counter
    call CalcSum             ; recursive call
L2: ret
CalcSum ENDP
```

Stack frame:

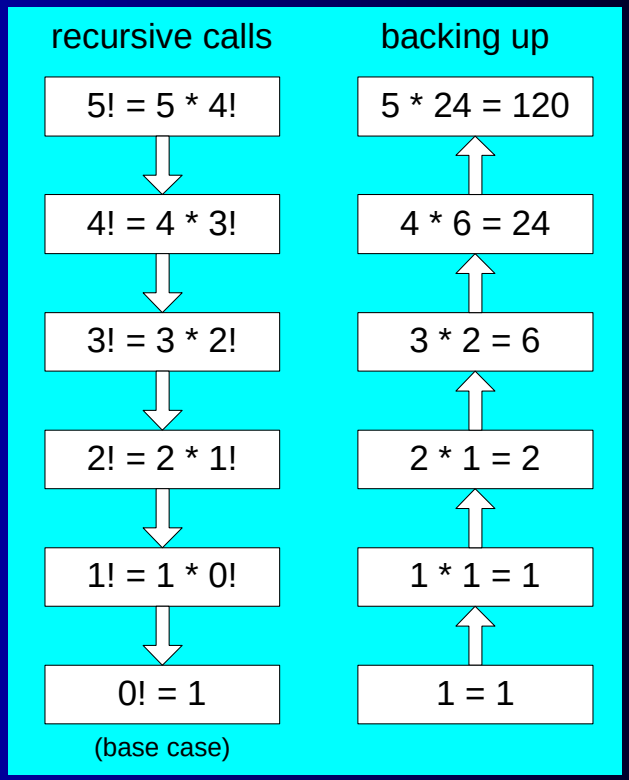
Pushed On Stack	ECX	EAX
L1	5	0
L2	4	5
L2	3	9
L2	2	12
L2	1	14
L2	0	15

View the complete program

Calculating a Factorial (1 of 3)

This function calculates the factorial of integer *n*. A new value of *n* is saved in each stack frame:

```
int function factorial(int n)
{
    if(n == 0)
      return 1;
    else
      return n * factorial(n-1);
}
```



As each call instance returns, the product it returns is multiplied by the previous value of n.

Calculating a Factorial (2 of 3)

```
Factorial PROC
    push ebp
    mov  ebp,esp
    mov  eax,[ebp+8]              ; get n
    cmp  eax,0                    ; n < 0?
    ja   L1                       ; yes: continue
    mov  eax,1                    ; no: return 1
    jmp  L2

L1: dec  eax
    push eax                      ; Factorial(n-1)
    call Factorial

; Instructions from this point on execute when each
; recursive call returns.

ReturnFact:
    mov  ebx,[ebp+8]              ; get n
    mul  ebx                      ; eax = eax * ebx

L2: pop  ebp                      ; return EAX
    ret  4                        ; clean up stack
Factorial ENDP
```

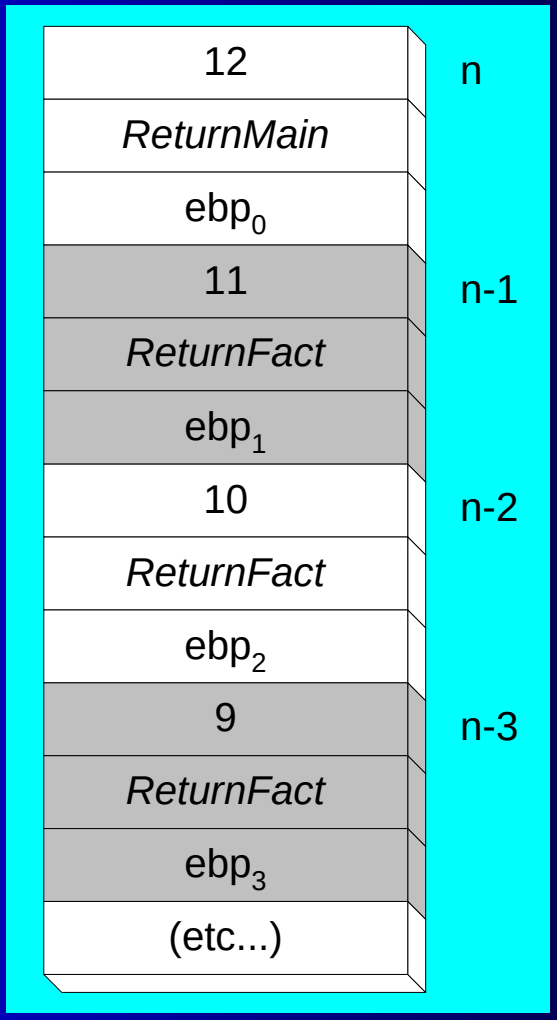
See the program listing

Calculating a Factorial (3 of 3)

Suppose we want to calculate 12!

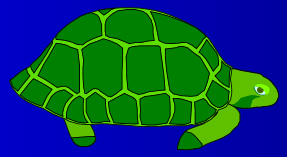
This diagram shows the first few stack frames created by recursive calls to Factorial

Each recursive call uses 12 bytes of stack space.



Summary

- Stack parameters
  - more convenient than register parameters
  - passed by value or reference
  - ENTER and LEAVE instructions
- Local variables
  - created on the stack below stack pointer
  - LOCAL directive
- Recursive procedure calls itself
- Calling conventions (C, stdcall)
- MASM procedure-related directives
  - INVOKE, PROC, PROTO



53 68 75 72 79 6F